LOT Pager ??? 05-LOT-API-CONNECTOR

05 · LOT API Connector — Device ⇄ lot-systems.com

Status: **proposed spec** (no server code written yet). Existing relevant code: `src/server/routes/api.ts` (`GET /logs`, `displayableEvents` list), `src/server/routes/public-api.ts` (`/status`, weather helper), AI engines in `src/server/utils/ai-engines.ts`.

1. Flow

```
AI engine (site) --decides nudge--> device_notifications table
      |                                       |
 WebSocket push / poll  <--------------------+
      v
 [LOT Pager screen: "Coffee time!"]
      | button = Copy
      v
 POST /api/device/event {type:"copy"} --> Log entry event "device_copy" --> Log tab
```

2. Auth

- Pairing: user opens Settings → Devices → "Add device" → 6-digit code (5 min TTL). Device calls `POST /api/device/pair {code, hw_id, fw}` → returns `device_token`.
- All other calls: `Authorization: Bearer <device_token>`. Tokens revocable per device.

3. Endpoints (all JSON, `/api/device/…`)

Method	Path	Purpose
POST	`/pair`	Exchange code → token
GET	`/inbox?since=<id>`	Pending notifications (long-poll up to 25 s)
WS	`/ws`	Push of notifications (preferred when powered/awake)
POST	`/ack` `{id, state:"shown"|"read"}`	Delivery receipts
POST	`/event`	Button / device events: `{type:"copy", notification_id, text, ts}`
POST	`/telemetry`	Batch: temp, rh, pressure, gas_class, lux, battery %, rssi, fw
POST	`/camera`	(opt-in) JPEG/embedding upload
GET	`/config`	Quiet hours, brightness, sensor cadence, OTA url

Notification object:


```
        {"id":"n_01","title":"Coffee time!","body":"You've been focused
2h.","priority":"nudge","ttl_s":3600,"created":"2026-10-27T13:30:00Z"}
```


4. Log-tab integration (button → Log)

1. Add `'device_copy'` (and `'device_telemetry'`, optional) to `displayableEvents` in `GET /logs` (`api.ts` ≈ line 1083).
2. `/event` handler creates `Log` row: `event: 'device_copy'`, `context: { device_id, notification_id, text }`, user = token owner.
3. Client Log tab renders it as "⎘ Copied from LOT Pager — Coffee time!".

5. AI generation

A scheduled job (see `src/server/scheduled-jobs.ts`) asks the AI engine for ≤ 80-char nudges using user's context (time of day, weather from `utils/weather`, Memory Engine); rate-limited (default ≤ 6/day, respects quiet hours); writes to the `device_notifications` table; pushes to WS.

6. Data model (Prisma/Sequelize)

`Device(id, userId, hwId, name, tokenHash, fw, lastSeen, createdAt, revokedAt)` · `DeviceNotification(id, userId, deviceId?, title, body, priority, status, createdAt, shownAt, readAt)` · `DeviceTelemetry(id, deviceId, ts, payload JSONB)`.

7. Next steps (HW-02)

Write migration + routes + simulator script (`scripts/device-sim.ts`) so the loop is testable before hardware exists; add Settings → Devices UI.